



IPT REFACTORING CASE STUDY

Refactoring the Naive Coffee Shop

Enforcing High Cohesion and Loose Coupling in Software Architecture

Prepared By Salvador's Team (BSIT 3A):

Diane Salvador • Lyra Andres • Jea Baluga • Melvin Apatas • Mycha Saggob



Core Principles of Clean Design



WHAT ARE COHESION & COUPLING?

In classical Software Engineering, modularity determines the system's longevity and adaptability. Our refactoring objective is to align with these two

pillars: **Cohesion:** How focused a single class's responsibilities are. We aim for high cohesion (one class does one job).

 **Coupling:** The degree of direct dependencies between classes. We aim for loose coupling.

MATHEMATICAL EVALUATION OF DESIGN

We can conceptually represent cohesive metric density using methods sharing attributes:

$$\text{Cohesion} = \frac{\sum A_i}{M \times A}$$

Low sharing represents cohesive decay, indicating that the class should be broken down into separate objects.





PART 01. The Naive Base Code

Deconstructing the structural traps inside our legacy application code



Legacy "Naive" Monolithic Code Structure



SPAGHETTI CODE UNDER THE MICROSCOPE

The original Base Code initialized everything inside a single nested execution tree. Observe the coupling inside our legacy pattern:

-  **Rigid Instantiation:** Directly calling new MySQLDatabase() inside the checkout loop.
-  **Mixed UI Logic:** Outputting raw formatted strings to the system console side-by-side with tax calculation math.

Result: Modifying print parameters broke database operations.

```
NaiveCoffeeShop.java

public class NaiveCoffeeShop {
    public void checkout(Order o) {
        // Hardcoded Database Dependency
        MySQLDatabase db = new MySQLDatabase();
        db.save(o);

        // Hardcoded calculations mixed with UI printing
        double tax = o.total() * 0.12;
        System.out.println("Receipt Total: " + (o.total() + tax));
    }
}
```



Three Core Structural Failures



Tight Coupling

The code is tightly locked to MySQL. Transitioning to local text files or MongoDB requires rewriting the entire system's execution core.



Cohesion Decay

The checkout class holds three unrelated reasons to change: database structures, tax calculation percentages, and console receipt print formats.



Untestable State

Since the database is constructed inline, we cannot test calculations without maintaining an active database connection. Mocking is impossible.





PART 02. The Refactoring Strategy

Extracting interfaces, isolating responsibilities, and enabling testability



How We Decoupled the System



ENFORCING DEPENDENCY INJECTION (DI)

Instead of classes declaring and building their own dependent objects, we modified our classes to demand their interfaces via constructor parameters.



Extract Interfaces: We created clean contracts like `OrderRepository` and `TaxCalculator`.



Manual Dependency Injection: We centralized object assembly inside `Main.java`.

ARCHITECTURAL LINEAGE COMPARISON

Legacy Pipeline:

```
Main → NaiveCoffeeShop (instantiates MySQL, Tax, UI directly)
```

Refactored Pipeline:

```
Main → Instantiates Interfaces → Injects them to OrderController
```



The Refactored "Clean" Implementation



OrderController.java

```
public class OrderController {  
    private final OrderRepository db;  
    private final TaxCalculator calc;  
    private final ReceiptPrinter printer;  
  
    // Loosely Coupled Constructor DI  
    public OrderController(OrderRepository r, TaxCalculator tc,  
        ReceiptPrinter rp) {  
        this.db = r;  
        this.calc = tc;  
        this.printer = rp;  
    }  
  
    public void checkout(Order o) {  
        db.save(o);  
        double total = calc.calculateTax(o.total());  
    }  
}
```

MODULAR CLEAN CODE BREAKDOWN

Observe how clean OrderController is now. It has absolutely zero knowledge of how the database saves the file or how the printer renders characters:

- 🛡️ **Safe Architecture:** Database type changes (from MySQL to Mock) require absolutely zero modifications to OrderController.
- ✅ **Isolated Testing:** We can now test the checkout execution tree with simple mocks.



Modular Responsibility Mapping



Tax Calculation

Calculates exact taxes safely. Completely decoupled from UI printing. Easily swap tax coefficients for municipal or country-wide rates.



Data Persistence

Handles reading, writing, and storage procedures. The system remains completely agnostic of the underlying database engine.



Output Generation

Handles printing and user representation. Formatting strings, layout shifts, or HTML output can be updated without breaking core application rules.



Architecture Improvement Benchmarks



By extracting decoupled services, we eliminated system-wide modification ripple errors, guaranteeing easy maintenance.



Teammate Roles & Contributions (100% Total)



To maintain maximum efficiency and ownership, each team member took complete responsibility for a key architectural segment:

Team Member	Class Asset / Module Owned	Specific Implementation Details	Contribution %
Diane Salvador	ReceiptPrinter.java	Designed clean output formatting, console receipt separation, and visual presentation layer rules.	20%
Lyra Andres	TaxCalculator.java	Engineered decoupled logic calculations and isolated numeric computation strategies.	20%
Jea Baluga	OrderRepository.java	Built database abstraction simulated drivers and structured object serialization logic.	20%
Melvin Apatas	OrderController.java	Designed dependency constructor injection routes and controlled modular service flow execution.	20%
Mycha Saggob	Main.java	Implemented the entry point engine, manual instantiation pipelines, and managed final assembly operations.	20%





Thank You!

Questions & Discussion - Let's talk Software Clean Architecture

BSIT 3A • Salvador's Team • IPT Refactoring Presentation

